

Processor Verification - Lab 4

Definition Specification Document

Lorenzo Pattaro Zonta

May 2, 2026

1 Fetch Unit

1.1 Functional Description

The fetch unit maintains a program counter (PC) and outputs the instruction located at the current PC address. It supports normal sequential execution and conditional branching via an alternate branch address.

1.2 Module Interface

Signal	Direction	Width	Description
clk	input	1	Clock signal.
rst_n	input	1	Active-low synchronous reset.
pc_en	input	1	Enables PC increment (sequential fetch).
branch_en	input	1	Enables branch to branch_addr.
branch_addr	input	8	Branch target address.
instr	output	16	Instruction word fetched from memory.

1.3 Behavioral Description

An instruction memory is initialized with dummy values. On each clock edge, the PC updates to either the branch address (if branch is enabled), the next sequential address ($PC + 2$ if `pc_en` is asserted), or holds its value. The output instruction is read from memory at the updated PC. On reset, the PC is cleared to zero and the instruction at address zero is output.

1.4 List of assertions

This is the list of the assertions for the fetch unit:

- Reset: after a reset cycle, PC must be zero

```
property p_reset_pc_zero;
    @(posedge clk) !rst_n | => (dut.pc == 8'h00);
endproperty
assert property (p_reset_pc_zero)
    else $error("PC not zero after reset");
```

- ```
property p_branch_priority_load;
 @(posedge clk) disable iff (!rst_n)
 $past(branch_en) |-> (dut.pc == $past(branch_addr));
endproperty

assert property (p_branch_priority_load)
 else $error("Branch priority load failed");
```

- ```

property p_branch_priority_not_increment;
@ (posedge clk) disable iff (!rst_n)
    $past(branch_en) |-> (dut.pc != ($past(dut.pc) + 8'd2));
endproperty

assert property (p_branch_priority_not_increment)
    else $error("Branch priority not-increment failed");

```
- ```

property p_pc_increment;
@ (posedge clk) disable iff (!rst_n)
 ($past(pc_en) && !$past(branch_en)) |-> (dut.pc == ($past(dut.pc) + 8'd2));
endproperty
assert property (p_pc_increment)
 else $error("PC increment failed");

```
- ```

property p_pc_hold;
@ (posedge clk) disable iff (!rst_n)
    (!$past(pc_en) && !$past(branch_en)) |-> (dut.pc == $past(dut.pc));
endproperty

assert property (p_pc_hold)
    else $error("PC hold failed");

```
- ```

property p_instr_matches_mem;
@ (posedge clk) disable iff (!rst_n)
 (instr == dut.mem[dut.pc]);
endproperty

assert property (p_instr_matches_mem)
 else $error("Instruction does not match memory at PC");

```
- ```

property p_pc_in_range;
@ (posedge clk) disable iff (!rst_n)
    (dut.pc inside {[8'h00:8'hFF]});
endproperty
assert property (p_pc_in_range)
    else $error("PC out of range");

```

2 Decode Unit

2.1 Functional Description

The decode unit receives a 16-bit instruction and splits it into opcode, destination register, source register, and immediate fields. It models a fixed two-cycle decode latency and performs a simple data hazard check that can stall decoding.

2.2 Module Interface

Signal	Direction	Width	Description
clk	input	1	Clock signal.
rst_n	input	1	Active-low synchronous reset.
instr_valid	input	1	Indicates the input instruction is valid.
instr	input	16	Instruction word from fetch stage.
decode_done	output	1	Pulses when decode latency completes.
opcode	output	4	Opcode field (bits 15:12).
rd	output	4	Destination register (bits 11:8).
rs	output	4	Source register (bits 7:4).
imm	output	4	Immediate field (bits 3:0).
hazard_stall	output	1	High when a data hazard is detected.

2.3 Behavioral Description

Instruction acceptance occurs when `instr_valid=1` and `hazard_stall=0` in the next cycle. On acceptance, the fields `opcode`, `rd`, `rs`, and `imm` capture `instr[15:12]`, `instr[11:8]`, `instr[7:4]`, and `instr[3:0]`, respectively. After an instruction is accepted in cycle N , `decode_done` must assert exactly in cycle $N + 2$ and it must be a single-cycle pulse.

A hazard is detected when the destination register `rd` of the previously accepted instruction matches the source register `rs` of the current instruction. When this hazard occurs, `hazard_stall` asserts, the current instruction is not accepted until the stall clears, and `decode_done` must not assert during the stall window. While `instr_valid=1` and `hazard_stall=1`, the decoded fields (`opcode`, `rd`, `rs`, `imm`) must retain their previous values and only update upon acceptance.

For back-to-back valid instructions with no hazard, each instruction is accepted immediately in its cycle, and each acceptance produces a `decode_done` pulse two cycles later, which may result in consecutive `decode_done` pulses.

When `rst_n=0`, all outputs are cleared to zero, and no instruction is accepted or completed until reset is released.

3 Conclusion

4 Appendix

4.1 Number of hours required

For the completion of this laboratory, it has been needed x hours.

4.2 Appendix A